

STATE MIND

Symbiotic Core V2 ThreeFAadapter

30-06-2026 – 09-07-2026



1. Project brief	2
2. Finding severity breakdown	3
3. Summary of findings	4
4. Conclusion	4
5. Findings report	5

Informational	ThreeFAdapter.isValidSignature() uses memory-based signature checker on calldata input	5
	ThreeFAdapter.getMaxAssets() omits maxAssetsPerRequest cap despite NatSpec	5
	ThreeFAdapter.onRequestConsumed() doesn't validate offer and signature	6
	Malicious request can inflate ThreeFAdapter.totalAssets() and extract vault liquidity at an overstated share price	7
	ThreeFAdapter.totalAssets() unguarded external call causes vault-wide DoS	8
	Yield recognized as a discrete step at request maturity lets zero-duration (JIT) depositors snipe other depositors' realized 3F yield	8
	ThreeFAdapter.finalizeRequest() cross-contract (read-only) reentrancy	9

1. Project brief



Title	Description
Client	Symbiotic
Project name	Symbiotic Core V2 ThreeFAdapter
Timeline	30-06-2026 - 09-07-2026

Project Log

Repository	https://github.com/symbioticfi/core-mirror	
Date	Commit Hash	Note
01-07-2026	b9fa1d309531a078299c4ce4f3c4bbd3a7b6478d	Initial Commit
06-07-2026	3b1a9327e3a3e668a2c90006b02c5ba926130fa5	Reaudit commit
08-07-2026	d18d4699c6a48f1efad8d7904a6e3427cd0dfd5a	Reaudit commit 2
09-07-2026	83a3a9ab8febe868896780a68345dbf820d843ff	Reaudit commit 3

Short Overview

The ThreeFAdapter is a Vault V2 adapter designed to integrate Symbiotic Core V2 with 3F bridge facilitator requests. The adapter represents a dedicated asset deployment layer through which Vault V2 liquidity can be allocated to 3F Request contracts, enabling the vault to participate in short-term bridge financing flows.

3F, through its Grunt protocol, enables leveraged exposure to asynchronously settled assets, such as tokenized treasuries, RWA funds, and other off-chain-settled instruments. Since such assets cannot be settled atomically within a single transaction, 3F introduces a bridge facilitation model where short-term capital is advanced to cover the settlement gap between a user’s deposit and the later receipt of the underlying asset. Once the asset settles and can be posted as collateral, the protocol borrows against it and repays the bridge facilitator.

Project Scope

The audit covered the following files:

 [ThreeFAdapter.sol](#)

2. Finding severity breakdown



All vulnerabilities discovered during the audit are classified based on their potential severity and have the following classification:

Severity	Description
Critical	Bugs leading to assets theft, fund access locking, or any other loss of funds to be transferred to any party.
High	Bugs that can trigger a contract failure. Further recovery is possible only by manual modification of the contract state or replacement.
Medium	Bugs that can break the intended contract logic or expose it to DoS attacks, but do not cause direct loss of funds.
Informational	Bugs that do not have a significant immediate impact and could be easily fixed.

Based on the feedback received from the Client regarding the list of findings discovered by the Contractor, they are assigned the following statuses:

Status	Description
Fixed	Recommended fixes have been made to the project code and no longer affect its security.
Acknowledged	The Client is aware of the finding. Recommendations for the finding are planned to be resolved in the future.

3. Summary of findings

Severity	# of Findings
Critical	0 (0 fixed, 0 acknowledged)
High	0 (0 fixed, 0 acknowledged)
Medium	0 (0 fixed, 0 acknowledged)
Informational	7 (4 fixed, 3 acknowledged)
Total	7 (4 fixed, 3 acknowledged)

4. Conclusion

During the audit of the codebase, 7 issues were found in total:

- 7 informational severity issues (4 fixed, 3 acknowledged)

The final reviewed commit is 83a3a9ab8febe868896780a68345dbf820d843ff

5. Findings report



INFORMATIONAL-01	ThreeFAdapter.isValidSignature() uses memory-based signature checker on calldata input	Fixed at: ee3e1b6
------------------	--	-----------------------------------

Description
Line: [ThreeFAdapter.sol#L67](#)
ThreeFAdapter.isValidSignature() accepts **bytes calldata signature** per IERC-1271 but delegates verification to **SignatureChecker.isValidSignatureNow()**, which takes **bytes memory**. Solidity copies the full signature into memory on every call. OpenZeppelin provides **SignatureChecker.isValidSignatureNowCalldata()** for this pattern; for an EOA **offerSigner** it uses **ECDSA.tryRecoverCalldata()** and avoids that copy.

Recommendation
We recommend switching to the calldata-aware helper so signature bytes are verified in place:

```
- return SignatureChecker.isValidSignatureNow(offerSigner, hash, signature)
+ return SignatureChecker.isValidSignatureNowCalldata(offerSigner, hash, signature)
  ? IERC1271.isValidSignature.selector
  : bytes4(0xffffffff);
```

INFORMATIONAL-02	ThreeFAdapter.getMaxAssets() omits maxAssetsPerRequest cap despite NatSpec	Fixed at: 8ed267b
------------------	--	-----------------------------------

Description
Line: [ThreeFAdapter.sol#L99-L102](#)
ThreeFAdapter.getMaxAssets() returns only **min(limitOf – totalAssets(), withdrawable())** and never applies **maxAssetsPerRequest**, even though **IThreeFAdapter.getMaxAssets()** documents the return value as the maximum principal fundable into the next request. **ThreeFAdapter.onRequestConsumed()** still reverts with **TooLargeRequest()** when **principalAssets > maxAssetsPerRequest**, so the view can report capacity the adapter will reject.

Recommendation
We recommend capping the return value with the configured per-request limit:

```
return Math.min(
  Math.min(
    IUniversalDelegator(delegator).limitOf(address(this)).saturatingSub(totalAssets()),
    IVaultV2(vault).withdrawable()
  ),
  maxAssetsPerRequest
);
```

INFORMATIONAL-03

ThreeFAdapter.onRequestConsumed() doesn't validate offer and signature

Fixed at:
[3b1a932](#)

Description

Line: [ThreeFAdapter.sol#L130](#)

ThreeFAdapter.onRequestConsumed() accepts **Offer** and **signature** but never uses them; principal release is authorized only by **msg.sender** passing the request whitelist, per-request amount/yield bounds, and **UniversalDelegator.allocateExact()**. Offer signing is expected to be enforced inside each 3F request contract before the callback, yet those requests are beacon proxies whose implementation can be swapped by the beacon owner. A whitelisted request address therefore keeps adapter trust after an upgrade even if the new logic skips **ThreeFAdapter.isValidSignature()**, nonce/expiry checks, or **offer.amount** consistency and calls **IThreeFRequest.onRequestConsumed()** with **principalAssets** up to **maxAssetsPerRequest**. Impact: Compromise or malicious action by the beacon owner can turn every whitelisted request into an unauthenticated JIT funding trigger without changing adapter configuration or **offerSigner**. Up to **MAX_REQUESTS** (50) concurrent requests may each pull **maxAssetsPerRequest**, draining a large fraction of the adapter's delegator limit in a single upgrade window before the whitelist can be paused.

Recommendation

We recommend enforcing offer authorization inside the adapter before allocating principal: verify the offer digest via **ThreeFAdapter.isValidSignature()**, check **offer.expiration** and **offer.nonce**, and require **offer.amount >= principalAssets** (and consistent yield fields).

Client's comments

Partially fixed

INFORMATIONAL-04	Malicious request can inflate ThreeFAdapter.totalAssets() and extract vault liquidity at an overstated share price	Acknowledged
------------------	---	--------------

Description

Lines: [ThreeFAdapter.sol#L83-L86](#)
When **IThreeFRequest.canWithdraw()** is true, **ThreeFAdapter.totalAssets()** values each request from external **IThreeFRequest.balancesOf()** and **IThreeFRequest.convertToAssets()** instead of adapter-held state. A malicious request can return inflated principal and yield while holding little or no collateral. That valuation feeds **VaultV2.getAccrueInterest()** and ERC-4626 share pricing on every deposit, withdraw, and redeem. An attacker can deposit at the pre-inflation price, trigger or wait for the inflated mark, then redeem against withdrawable vault liquidity at the overstated share price.
Impact: A malicious request can overstate adapter NAV and let a coordinated depositor withdraw more underlying than was contributed. The inflated withdrawn amount is bounded only by available withdrawable liquidity.

Recommendation

We recommend capping each request's contribution to values fixed at consumption time.
ThreeFAdapter.onRequestConsumed() already receives **principalAssets** and **yieldAssets**; persist both (e.g. **pendingPrincipal** and **pendingYield**) and use them as the book ceiling in **ThreeFAdapter.totalAssets()**:

```
if (IThreeFRequest(request).canWithdraw()) {
  (uint256 ptShares, uint256 ytShares) = IThreeFRequest(request).balancesOf(address(this));
  (uint256 pAssets, uint256 yAssets) = IThreeFRequest(request).convertToAssets(ptShares, ytShares);
  - assets += pAssets + yAssets;
  + assets += Math.min(pAssets + yAssets, pendingPrincipal[request] + pendingYield[request]);
} else {
  assets += pendingAssets[request];
}
```

Alternatively, keep using **pendingAssets** plus a stored **pendingYield** until **ThreeFAdapter.finalizeRequest()** realizes actual **IThreeFRequest.burnAll()** proceeds, so live request oracles cannot move vault share price above the adapter's recorded loan terms.

Client's comments

We should not limit yield with pendingYield, so currently leaving as-is.

INFORMATIONAL-05	ThreeFAdapter.totalAssets() unguarded external call causes vault-wide DoS	Acknowledged
------------------	--	--------------

Description

Line: [ThreeFAdapter.sol#L78](#)

ThreeFAdapter.totalAssets() iterates every tracked request (up to **MAX_REQUESTS=50**) and per request makes unguarded external static calls: **request.canWithdraw()**, and when true **request.balancesOf() + request.convertToAssets()**. None are wrapped in try/catch or gas-bounded. This is the vault-wide accounting hot path: **VaultV2.getAccrueInterest()** sums **UniversalDelegator.totalAssets() = Sum(IAdapter.totalAssets())**, and **VaultV2.accrueInterest()** runs on every deposit/withdraw/redeem/pull.

A single tracked request that reverts (or is deliberately gas-heavy) makes **ThreeFAdapter.totalAssets()** revert, reverting **VaultV2.accrueInterest()** and bricking all deposits/withdrawals/allocations for the entire vault and every other adapter under the same delegator. Recovery is impossible: the only eviction path **ThreeFAdapter.finalizeRequest()** itself calls **request.burnAll()** on the same adversarial contract (can be made to revert), and **UniversalDelegator.removeAdapter()** reads the reverting **ThreeFAdapter.totalAssets()** and reverts on **AdapterHasAssets**. There is no admin force-remove. Impact: Permanent, unrecoverable denial of service of the entire vault (all deposits, withdrawals, allocations, and fee accrual revert), locking all vault funds; the trigger is a single misbehaving whitelisted request.

Recommendation

We recommend to wrap each per-request external call in try/catch, falling back to the conservative book value **pendingAssets[request]** on failure so one request cannot revert the aggregate. Add an owner/permissioned force-remove that clears **requests[]/requestIndex/pendingAssets** without invoking **request.burnAll()**.

Client's comments

Currently, seems better to revert on unexpected behavior than silently skip.

INFORMATIONAL-06	Yield recognized as a discrete step at request maturity lets zero-duration (JIT) depositors snipe other depositors' realized 3F yield	Acknowledged
------------------	--	--------------

Description

Line: [ThreeFAdapter.sol#L78](#)

ThreeFAdapter.totalAssets() values each request at book value **pendingAssets** while **canWithdraw()==false** and switches to mark-to-market **convertToAssets() = principal+yield** once **canWithdraw()==true**. The entire accrued yield of a request is therefore recognized as a single discrete jump at the maturity block, not streamed over its lifetime. **VaultV2.getAccrueInterest()** treats **interest = newTotalAssets - _totalAssets** as share-price appreciation to whoever holds shares at that instant, and **VaultV2** deposit/redeem are permissionless and instant. An attacker deposits **D** in the maturity block at the pre-jump price, the flip raises **totalAssets** by yield **Y**, a pro-rata slice **D/(T+D)** of **Y** accrues to the attacker, who redeems immediately. Yield that economically belonged to holders during the request lifetime is transferred to a zero-duration depositor who doesn't take on the 3F risk; repeatable at every maturity across up to 50 concurrent requests.

Recommendation

We recommend to not recognize a request's yield as an instantaneous step: stream/amortize expected yield over the request lifetime, or cap per-request contribution to book value until finalize and gate deposits/withdrawals with a snapshot or short lock so a zero-duration depositor cannot capture the step.

Client's comments

Acknowledged.

Description

Line: [ThreeFAdapter.sol#L185](#)
In **ThreeFAdapter.finalizeRequest()** the adapter mutates state before the external call **request.burnAll()**. The **nonReentrant** guard is per-contract; **VaultV2.deposit()** and **VaultV2.accrueInterest()** are not protected by the adapter's guard. During **request.burnAll()** – if a malicious/compromised whitelisted request hands control back before transferring the redeemed assets in – **ThreeFAdapter.totalAssets()** is transiently deflated: the finalized request is already removed from the loop while its redeemed assets have not yet landed in **ThreeFAdapter.freeAssets()**.
An attacker re-entering **VaultV2.deposit()** in that window mints shares at the deflated price; when **request.burnAll()** completes and assets arrive, **totalAssets** snaps back up and the attacker's inflated share count captures value from existing holders.

Recommendation

We recommend to add a cross-contract reentrancy lock shared with the vault flow.

STATE MIND